Conception et Mise en Œuvre d'une Infrastructure Virtualisée Sécurisée et Orientée Big Data

Projet de Fin de Module

Étudiant	Ayoub Lahlaibi et El Mehdi El khaldi
Hyperviseur	Oracle VirtualBox (Type 2)
OS Invités	Ubuntu Server 22.04 LTS (x3)
Année	2025 – 2026

Chapitre 0 — Résumé

Ce rapport présente la conception et la mise en œuvre d'une **infrastructure virtualisée complète, sécurisée et orientée Big Data**, réalisée dans le cadre du module de Virtualisation.

L'infrastructure repose sur **Oracle VirtualBox** comme hyperviseur de Type 2, hébergeant **trois machines virtuelles** Ubuntu Server 22.04 LTS aux rôles distincts : serveur applicatif, serveur de base de données et serveur d'administration.

La virtualisation réseau est assurée par **trois VLANs isolés** (VLAN 10, 20, 30) via des NAT Networks, un firewall virtuel **UFW** et un **Bastion Host** garantissant un point d'accès SSH unique.

La virtualisation du stockage intègre trois niveaux complémentaires : **DAS** (disques VDI avec Thin Provisioning et LVM), **NAS** (partage NFS avec quatre volumes) et **SAN** (stockage bloc iSCSI de 2 GB), avec snapshots et sauvegardes automatiques planifiées.

La containerisation repose sur **Docker Compose** pour déployer une stack Big Data complète : **Nginx**, **React.js**, **Node.js**, **MySQL 8.0** et **phpMyAdmin**.

Mots-clés : Virtualisation, VirtualBox, Docker, VLANs, NFS, iSCSI, Big Data, React.js, Node.js, MySQL, UFW, Bastion Host, SSH RSA 4096.

Table des matières

Résumé **1**

Liste des figures **6**

Liste des tableaux **7**

1 Introduction Générale **8**
- 1.1 Contexte et Problématique 8
- 1.2 Objectifs du Projet 8
- 1.3 Environnement Matériel 9
- 1.4 Organisation du Rapport 9

2 Architecture Globale de l'Infrastructure **10**
- 2.1 Vue d'Ensemble 10
- 2.2 Machines Virtuelles et Rôles 11
- 2.3 Flux de Communication 11

3 Virtualisation Serveur **12**
- 3.1 Hyperviseur VirtualBox 12
- 3.2 Configuration des Machines Virtuelles 13
- 3.3 Thin Provisioning 13
- 3.4 LVM — Logical Volume Manager 14
- 3.5 Snapshots VirtualBox 15

4 Virtualisation Réseau **16**
- 4.1 Architecture Réseau Globale 16
- 4.2 Segmentation par VLANs 16
  - 4.2.1 Preuve d'Isolation des VLANs 17
- 4.3 Réseaux VirtualBox 18
  - 4.3.1 NAT Networks — Routeurs Virtuels 18
  - 4.3.2 Réseau Host-Only (192.168.56.0/24) 18
  - 4.3.3 Réseau Internal Network (prod-net) 18
  - 4.3.4 Réseau Docker bridge (app-network) 19

4.4 Firewall Virtuel — UFW . . . 19
4.5 Bastion Host — Point d'Accès Unique . . . 19

5 Virtualisation du Stockage . . . 21
5.1 Architecture du Stockage . . . 21
5.2 DAS — Direct Attached Storage . . . 22
5.3 NAS — Network Attached Storage . . . 22
5.3.1 Serveur NFS sur vm-applicatif . . . 22
5.3.2 Clients NFS (montage automatique) . . . 23
5.3.3 Test de partage NAS . . . 24
5.4 SAN — Storage Area Network . . . 24
5.4.1 Serveur iSCSI sur vm-applicatif . . . 24
5.4.2 Client iSCSI sur vm-database . . . 25

6 Containerisation et Application Big Data . . . 27
6.1 Architecture Docker . . . 27
6.2 Application Big Data — Architecture 3-Tiers . . . 28
6.2.1 Page d'accueil Nginx . . . 29
6.2.2 Dashboard React.js . . . 29
6.2.3 API Backend Node.js . . . 30
6.2.4 Interface phpMyAdmin . . . 31
6.2.5 Logs Backend . . . 32
6.3 Démarrage Automatique avec systemd . . . 32

7 Sécurité de l'Infrastructure . . . 34
7.1 Approche Multi-Couches . . . 34
7.2 Chiffrement SSH RSA 4096 . . . 34
7.3 Sécurisation MySQL . . . 35
7.4 Contrôle d'Accès par VLAN . . . 35

8 Administration et Monitoring . . . 36
8.1 Rôles de vm-gestion . . . 36
8.2 Monitoring à Distance . . . 37
8.3 Sauvegarde Automatique . . . 38

9 Difficultés Rencontrées et Solutions . . . 39
9.1 Difficulté 1 — Erreur d'Installation Ubuntu . . . 39
9.2 Difficulté 2 — Conflit d'IPs après Clonage . . . 39
9.3 Difficulté 3 — Disque vm-applicatif Saturé . . . 40
9.4 Difficulté 4 — Erreur ContainerConfig Docker Compose . . . 40

9.5 Difficulté 5 — SAN iSCSI Non Monté au Démarrage . . . 41
9.6 Difficulté 6 — Backend Node.js Ne Se Connecte Pas à MySQL . . . 41

10 Analyse Critique . . . 42
10.1 Apports de la Virtualisation . . . 42
10.1.1 Optimisation des ressources . . . 42
10.1.2 Sécurité renforcée . . . 42
10.1.3 Flexibilité et résilience . . . 42
10.2 Apports de la Containerisation . . . 43

11 Conclusion . . . 44

Références Bibliographiques . . . 45

Table des figures

2.1 Schéma d'architecture globale — Infrastructure virtualisée Big Data . 10
2.2 Interface VirtualBox — Les trois VMs en fonctionnement simultané . 11

3.1 Hostname et informations système de chaque VM (hostnamectl) . . . 13
3.2 Thin Provisioning — Disque virtuel 60 GB occupant réellement 11 GB 13
3.3 Structure LVM sur vm-applicatif (pvdisplay, vgdisplay, lvdisplay) . . 14
3.4 Snapshots VirtualBox des trois VMs 15

4.1 Configuration des trois VLANs dans VirtualBox (NAT Networks) . . 16
4.2 IPs VLAN attribuées à chaque VM (ip a show enp0s3) 17
4.3 Isolation VLAN prouvée par 100% packet loss entre vm-applicatif et vm-database 17
4.4 Configuration UFW sur les trois VMs 19
4.5 Démonstration du Bastion Host 20

5.1 LVM sur vm-applicatif — Volumes physiques, groupes et logiques . . 22
5.2 Exports NFS du serveur NAS (exportfs -v) 22
5.3 NAS monté automatiquement sur vm-database (df -h | grep nas) . . . 23
5.4 Fichier créé sur vm-applicatif visible simultanément sur les trois VMs 24
5.5 Serveur SAN iSCSI — Target avec LUN de 2147 MB 25
5.6 Client SAN — Disque iSCSI /dev/sdb (2 GB) monté sur /mnt/san . 25
5.7 Test SAN — Écriture et lecture sur /mnt/san 26

6.1 Containers Docker actifs (docker-compose ps) 27
6.2 Page d'accueil Nginx avec liens vers tous les services 29
6.3 Dashboard Big Data — Interface React.js professionnelle 29
6.4 Ajout d'une donnée Big Data via le formulaire React.js 30
6.5 API Backend Node.js — Endpoint /api/status 30
6.6 phpMyAdmin — Visualisation de bigdata_db et de la table donnees . 31
6.7 Logs du backend Node.js — Connexion réussie à MySQL 32

7.1 SSH sans mot de passe depuis vm-gestion vers les autres VMs 35

8.1 Monitoring depuis vm-gestion — Docker, MySQL, disque, RAM . . . 37
8.2 Sauvegarde automatique MySQL dans /mnt/nas/backup/ 38

Liste des tableaux

1.1 Spécifications de la machine hôte 9

2.1 Description et configuration des trois VMs 11

3.1 Configuration matérielle des VMs 13
3.2 Snapshots créés 15

4.1 Configuration des VLANs 17
4.2 Règles UFW par VM 19

5.1 Récapitulatif des trois niveaux de stockage 21
5.2 Dossiers partagés via NFS 23

6.1 Stack Docker déployée 28

7.1 Mesures de sécurité implémentées 34

Chapitre 1 — Introduction Générale

1.1 Contexte et Problématique

Les entreprises modernes s'appuient massivement sur des infrastructures virtualisées pour optimiser leurs ressources, améliorer leur sécurité et accélérer le déploiement d'environnements applicatifs. Dans un contexte **Big Data**, ces infrastructures doivent être capables de gérer de grands volumes de données de manière performante, sécurisée et scalable.

Enjeux de la virtualisation en entreprise :

- Réduction des coûts matériels par consolidation des serveurs
- Amélioration de la flexibilité et de la disponibilité
- Isolation et sécurisation des environnements
- Facilité de déploiement et de sauvegarde

1.2 Objectifs du Projet

Ce projet vise à concevoir et mettre en œuvre une infrastructure virtualisée complète articulée autour de quatre piliers fondamentaux :

1. **Virtualisation serveur :** Déploiement d'un hyperviseur VirtualBox hébergeant trois VMs Ubuntu Server avec des rôles spécialisés.
2. **Virtualisation réseau :** Segmentation par VLANs, firewall virtuel UFW, Bastion Host et architecture Docker bridge.

3. **Virtualisation du stockage :** Implémentation de DAS, NAS (NFS) et SAN (iSCSI), avec Thin Provisioning, LVM, snapshots et sauvegardes automatiques.
4. **Containerisation :** Déploiement d'une stack applicative Big Data complète via Docker Compose avec démarrage automatique systemd.

1.3 Environnement Matériel

Table 1.1 – *Spécifications de la machine hôte*

tableheadbg	
Système d'exploitation	Windows 10 Pro 64-bit
tablerowalt Processeur	Intel/AMD 64-bit avec VT-x/AMD-V activé
Mémoire RAM	20 GB
tablerowalt Stockage disponible	SSD — 413 GB libres (C :) + 1 TO (D :)
Hyperviseur	Oracle VirtualBox (Type 2) — Version récente
tablerowalt Virtualisation matérielle	VT-x activé (vérifié via Gestionnaire de tâches)

1.4 Organisation du Rapport

Ce rapport est structuré en onze chapitres : après cette introduction, les chapitres 2 à 6 présentent l'architecture et les quatre types de virtualisation implémentés. Le chapitre 7 traite de la sécurité, le chapitre 8 de l'administration, le chapitre 9 des difficultés rencontrées, et les chapitres 10 et 11 concluent par une analyse critique et une conclusion générale.

Chapitre 2 — Architecture Globale de l'Infrastructure

2.1 Vue d'Ensemble

L'infrastructure conçue est organisée en couches superposées, chacune assurant une fonction spécifique dans l'écosystème virtualisé.

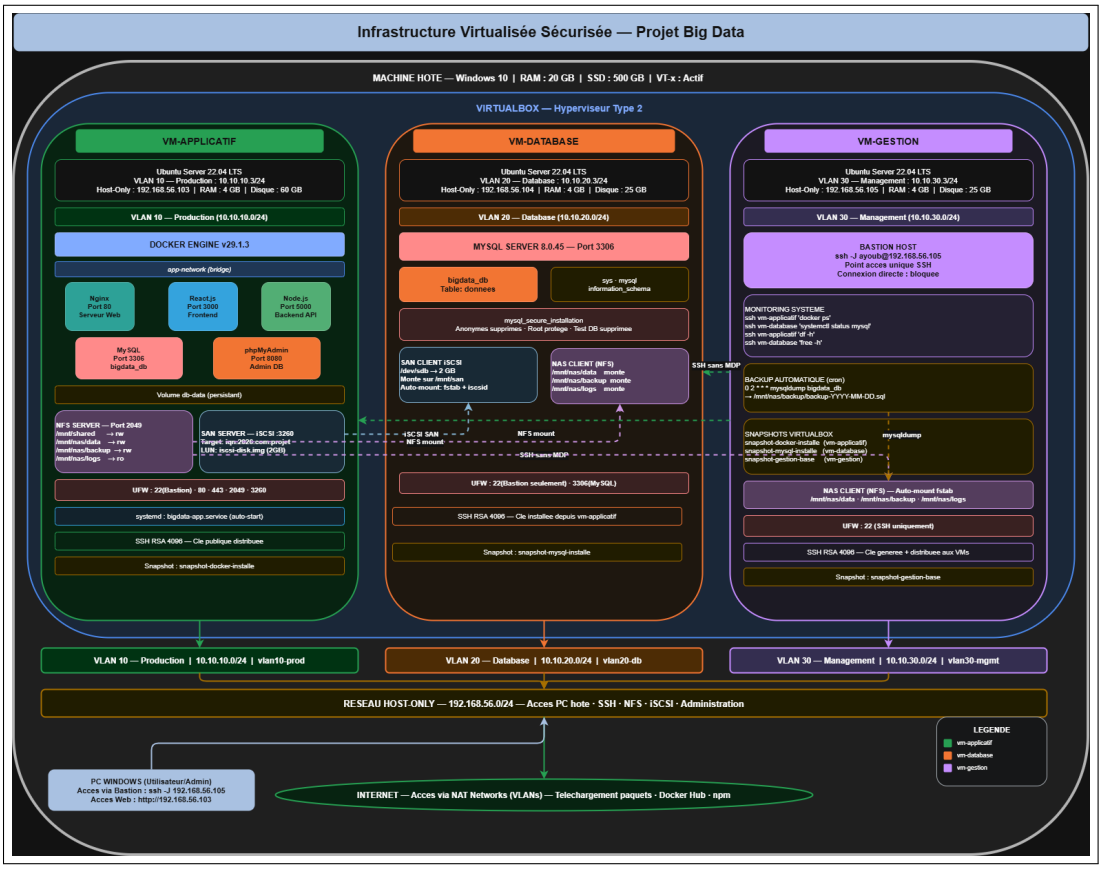


Figure 2.1 – *Schéma d'architecture globale — Infrastructure virtualisée Big Data*

2.2 Machines Virtuelles et Rôles

Table 2.1 – *Description et configuration des trois VMs*

tableheadbg				
vm-applicatif	Web + Docker + NAS + SAN	`10.10.10.3`	`192.168.56.103`	60 GB
tablerowalt **vm-database**	MySQL + Client SAN/NAS	`10.10.20.3`	`192.168.56.104`	25 GB
vm-gestion	Bastion + Monitoring + Backup	`10.10.30.3`	`192.168.56.105`	25 GB

2.3 Flux de Communication

Figure 2.2 – *Interface VirtualBox — Les trois VMs en fonctionnement simultané*

Chapitre 3 — Virtualisation Serveur

3.1 Hyperviseur VirtualBox

Oracle VirtualBox est un hyperviseur de **Type 2** (hosted) installé sur le système d'exploitation Windows hôte. Il utilise les extensions matérielles VT-x/AMD-V du processeur pour exécuter directement le code des VMs, offrant de bonnes performances malgré la couche d'abstraction supplémentaire.

Hyperviseur Type 2 vs Type 1
Un hyperviseur de **Type 1** (bare-metal) comme VMware ESXi s'installe directement sur le matériel, sans OS hôte. Un hyperviseur de **Type 2** comme VirtualBox s'installe sur un OS existant. Le Type 2 est moins performant mais plus accessible pour un environnement de développement ou académique.

3.2 Configuration des Machines Virtuelles

Table 3.1 – *Configuration matérielle des VMs*

tableheadbg				
vm-applicatif	2	4 GB	60 GB	Ubuntu Server 22.04 LTS
tablerowalt vm-database	2	4 GB	25 GB	Ubuntu Server 22.04 LTS
vm-gestion	2	4 GB	25 GB	Ubuntu Server 22.04 LTS

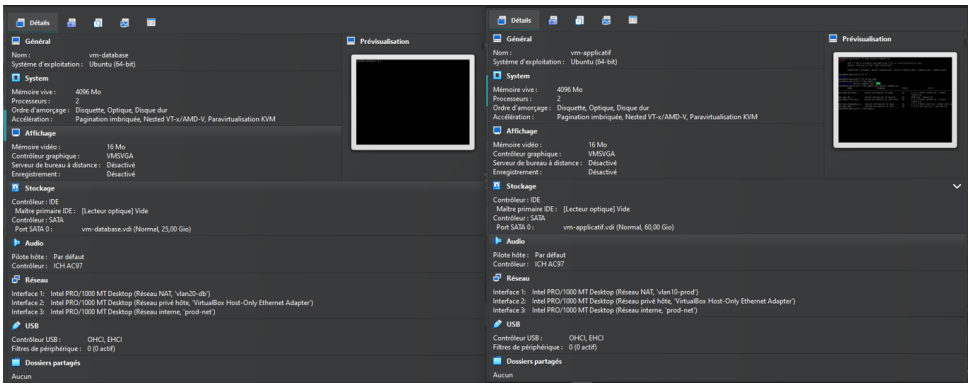


Figure 3.1 – *Hostname et informations système de chaque VM (hostnamectl)*

3.3 Thin Provisioning

Les disques VDI utilisent le **Thin Provisioning** (allocation dynamique) : l'espace disque est alloué progressivement selon les besoins réels.

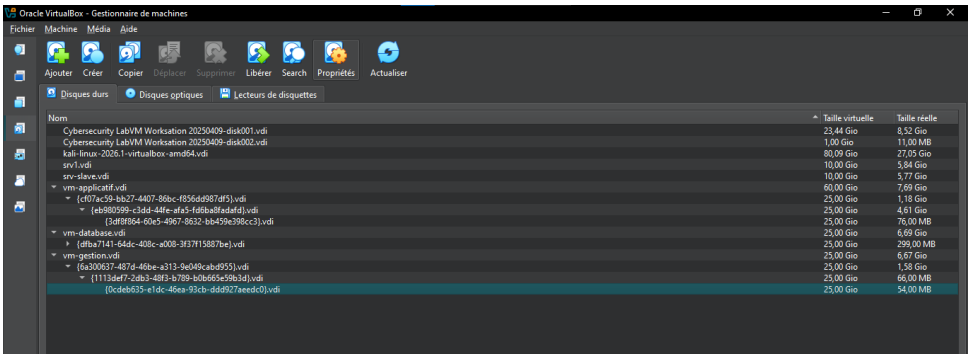


Figure 3.2 – *Thin Provisioning — Disque virtuel 60 GB occupant réellement 11 GB*

Avantage du Thin Provisioning : Un disque virtuel de 60 GB n'occupe que l'espace réellement utilisé sur le disque hôte (11 GB), économisant ainsi l'espace de stockage physique.

3.4 LVM — Logical Volume Manager

Ubuntu Server utilise **LVM** pour la gestion flexible des partitions, permettant l'extension à chaud des volumes logiques sans redémarrage.

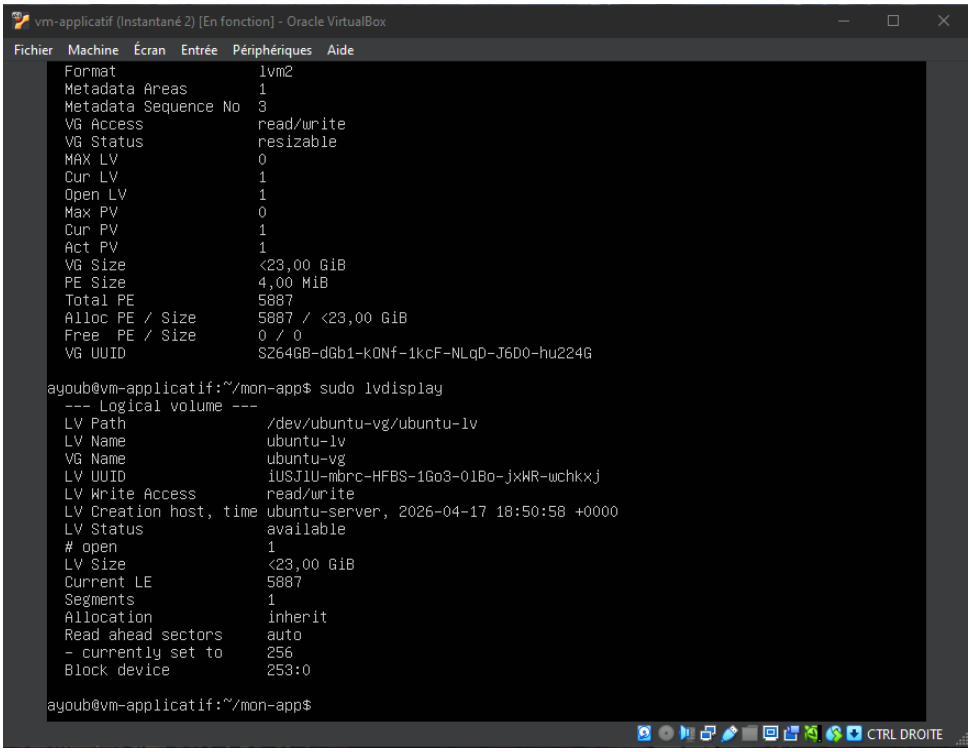


Figure 3.3 – *Structure LVM sur vm-applicatif (pvdisplay, vgdisplay, lvdisplay)*

3.5 Snapshots VirtualBox

Des instantanés ont été créés pour chaque VM afin de permettre une restauration rapide. Un snapshot capture l'état complet (disque + mémoire) à un instant donné.

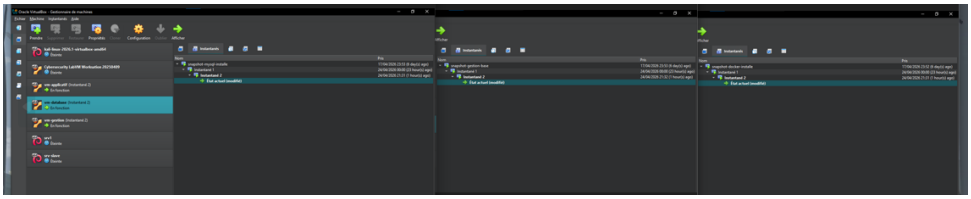


Figure 3.4 – *Snapshots VirtualBox des trois VMs*

Table 3.2 – *Snapshots créés*

tableheadbg		
snapshot-docker-installe	vm-applicatif	Après installation Docker
tablerowalt snapshot-mysql-installe	vm-database	Après installation MySQL
snapshot-gestion-base	vm-gestion	Configuration de base

Chapitre 4 — Virtualisation Réseau

4.1 Architecture Réseau Globale

La virtualisation réseau a été implémentée à plusieurs niveaux hiérarchiques, formant une architecture de sécurité en couches (defense in depth).

4.2 Segmentation par VLANs

Trois VLANs ont été créés dans VirtualBox via des **NAT Networks** séparés, chacun isolant une VM dans son propre segment réseau indépendant.

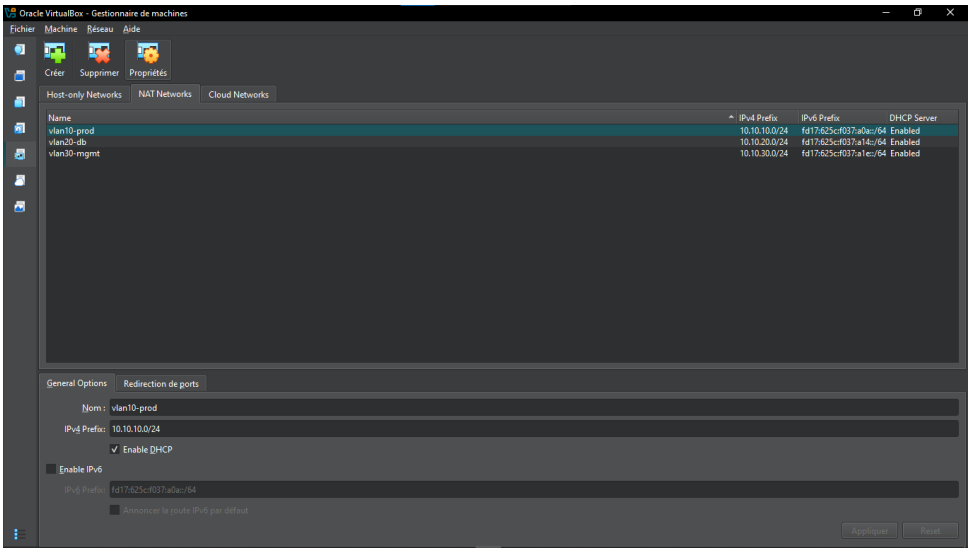


Figure 4.1 – *Configuration des trois VLANs dans VirtualBox (NAT Networks)*

Table 4.1 – *Configuration des VLANs*

tableheadbg			
VLAN 10	vlan10-prod	10.10.10.0/24	vm-applicatif (Production)
tablerowalt **VLAN 20**	vlan20-db	10.10.20.0/24	vm-database (Base de données)
VLAN 30	vlan30-mgmt	10.10.30.0/24	vm-gestion (Administration)

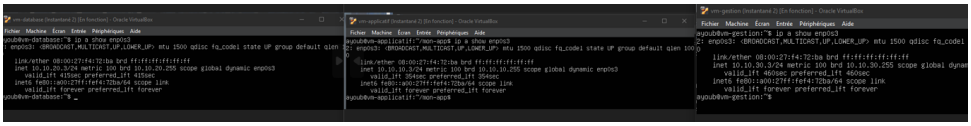


Figure 4.2 – *IPs VLAN attribuées à chaque VM (ip a show enp0s3)*

4.2.1 Preuve d'Isolation des VLANs

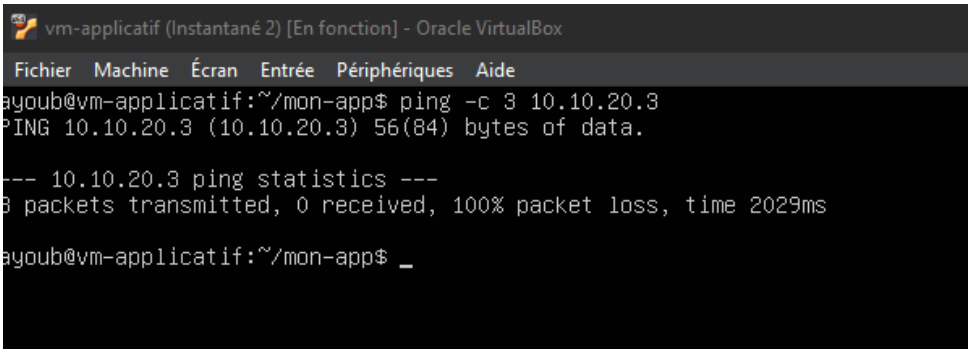


Figure 4.3 – *Isolation VLAN prouvée par 100% packet loss entre vm-applicatif et vm-database*

```
ayoub@vm-applicatif:~$ ping -c 3 10.10.20.3
PING 10.10.20.3 56(84) bytes of data.
--- 10.10.20.3 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2050
    ms
# Les VLANs sont correctement isoles
```

Listing 4.1 – *Test d'isolation VLAN — 100% packet loss*

4.3 Réseaux VirtualBox

4.3.1 NAT Networks — Routeurs Virtuels

Chaque VLAN dispose de son propre NAT Network qui joue le rôle de **routeur virtuel**. Ce routeur gère la traduction d'adresses (NAT) pour l'accès à internet et isole totalement le trafic inter-VLANs.

4.3.2 Réseau Host-Only (192.168.56.0/24)

Un réseau Host-Only connecte les VMs entre elles et avec le PC hôte, utilisé pour l'accès aux services web, NFS, iSCSI et l'administration SSH.

4.3.3 Réseau Internal Network (prod-net)

Un réseau interne assure la communication privée inter-VMs, totalement inaccessible depuis le PC hôte ou internet.

4.3.4 Réseau Docker bridge (app-network)

Docker crée automatiquement un réseau bridge virtuel connectant les cinq containers applicatifs de manière isolée.

4.4 Firewall Virtuel — UFW

UFW (Uncomplicated Firewall) joue le rôle de **pare-feu virtuel** sur chaque VM, avec des règles strictement adaptées au rôle de chaque machine.

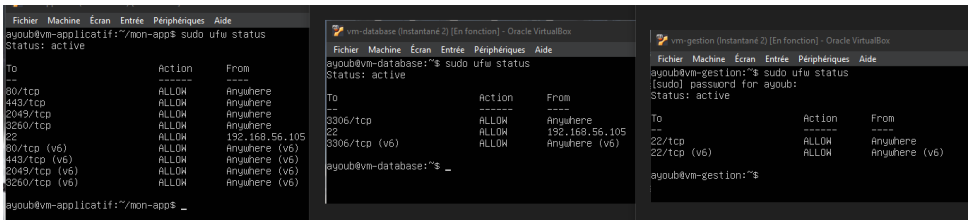


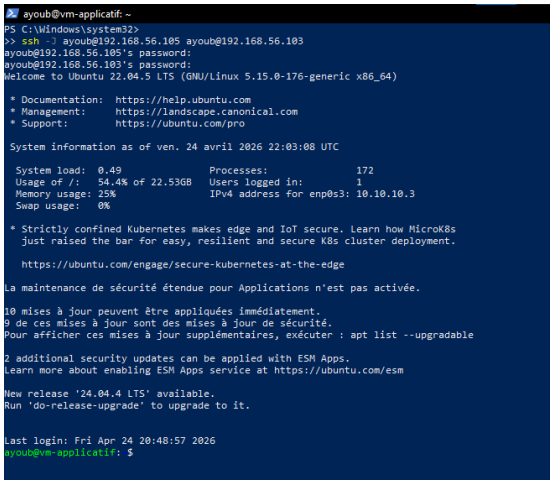
Figure 4.4 – *Configuration UFW sur les trois VMs*

Table 4.2 – *Règles UFW par VM*

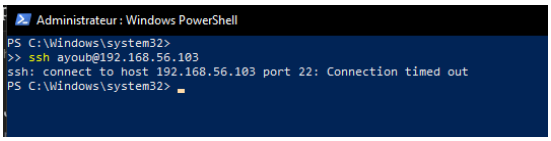
tableheadbg		
vm-applicatif	22 (Bastion), 80, 443, 2049, 3260	SSH restreint, Web, NFS, iSCSI
tablerowalt **vm-database**	22 (Bastion seulement), 3306	SSH depuis vm-gestion uniquement, MySQL
vm-gestion	22 (SSH)	Administration uniquement

4.5 Bastion Host — Point d'Accès Unique

vm-gestion joue le rôle de **Bastion Host** : toute connexion SSH aux VMs internes doit obligatoirement transiter par vm-gestion.



(a) *Connexion via Bastion — Succès*



(b) *Connexion directe — Bloquée*

Figure 4.5 – *Démonstration du Bastion Host*

```
# Via Bastion Host (succes)
PS C:\> ssh -J ayoub@192.168.56.105 ayoub@192.168.56.103
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-176-generic
    x86_64)
ayoub@vm-applicatif:~$

# Connexion directe (bloquee par UFW)
PS C:\> ssh ayoub@192.168.56.103
ssh: connect to host 192.168.56.103 port 22: Connection timed
    out
```

Listing 4.2 – *Connexion SSH via Bastion (succes) vs. directe (bloquee)*

Chapitre 5 — Virtualisation du Stockage

5.1 Architecture du Stockage

La virtualisation du stockage a été implémentée selon trois architectures complémentaires couvrant l'ensemble du spectre des solutions de stockage virtualisé en entreprise.

Table 5.1 – *Récapitulatif des trois niveaux de stockage*

tableheadbg				
DAS	Disques VDI	Direct/LVM	Chaque VM	OS et données locales
tablerowalt **NAS**	NFS	TCP port 2049	/mnt/nas/*	Partage inter-VMs
SAN	iSCSI	TCP port 3260	/mnt/san	Stockage dédié vm-database

5.2 DAS — Direct Attached Storage

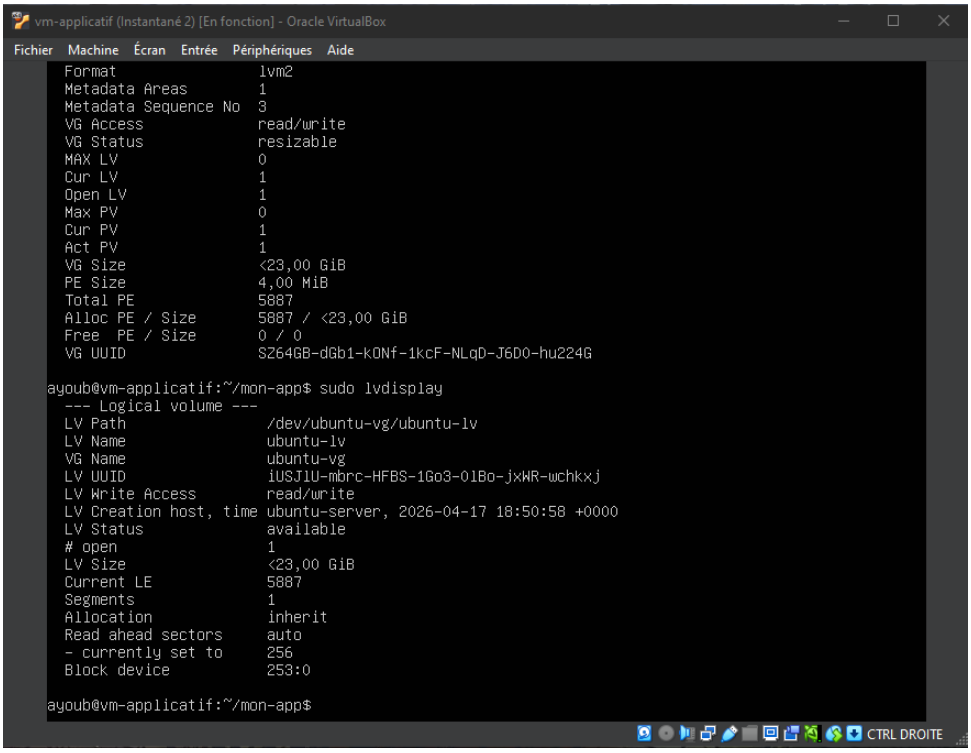


Figure 5.1 – *LVM sur vm-applicatif — Volumes physiques, groupes et logiques*

5.3 NAS — Network Attached Storage

5.3.1 Serveur NFS sur vm-applicatif

vm-applicatif joue le rôle de serveur NAS via le protocole NFS. Quatre dossiers sont exportés avec des permissions différenciées.

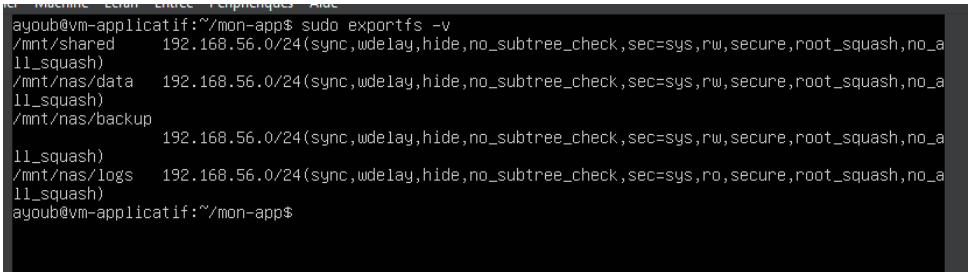


Figure 5.2 – *Exports NFS du serveur NAS (exportfs -v)*

Table 5.2 – *Dossiers partagés via NFS*

tableheadbg		
`/mnt/shared`	rw (lecture/écriture)	Partage général inter-VMs
tablerowalt `/mnt/nas/data`	rw (lecture/écriture)	Données partagées
`/mnt/nas/backup`	rw (lecture/écriture)	Sauvegardes MySQL automatiques
tablerowalt `/mnt/nas/logs`	ro (lecture seule)	Journaux système centralisés

5.3.2 Clients NFS (montage automatique)

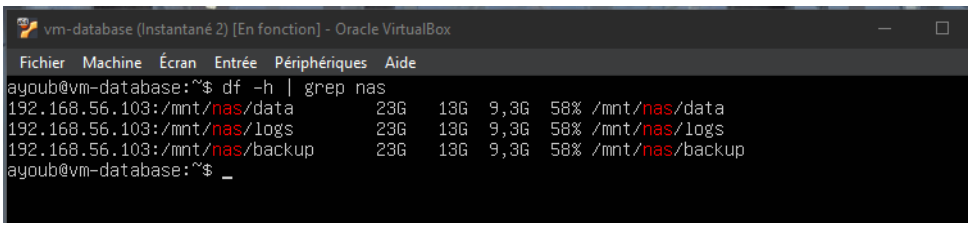


Figure 5.3 – *NAS monté automatiquement sur vm-database (df -h | grep nas)*

5.3.3 Test de partage NAS

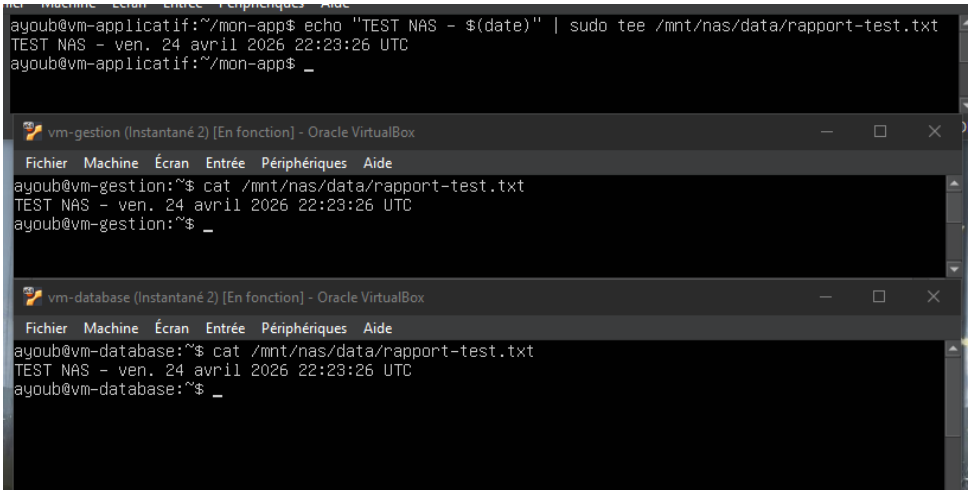


Figure 5.4 – *Fichier créé sur vm-applicatif visible simultanément sur les trois VMs*

```
# vm-applicatif (serveur) --- Creation
echo "TEST NAS - $(date)" | sudo tee /mnt/nas/data/test.txt
TEST NAS - jeu. 23 avril 2026 20:30:05 UTC

# vm-database (client) --- Lecture immediate
cat /mnt/nas/data/test.txt
TEST NAS - jeu. 23 avril 2026 20:30:05 UTC  [OK]

# vm-gestion (client) --- Lecture immediate
cat /mnt/nas/data/test.txt
TEST NAS - jeu. 23 avril 2026 20:30:05 UTC  [OK]
```

Listing 5.1 – *Test de partage NAS — Lecture simultanee sur les 3 VMs*

5.4 SAN — Storage Area Network

5.4.1 Serveur iSCSI sur vm-applicatif

vm-applicatif expose un disque virtuel de 2 GB via le protocole iSCSI (port 3260). Ce LUN (Logical Unit Number) est accessible depuis vm-database comme un disque local `/dev/sdb`.

```
        I_T nexus: 1
            Initiator: iqn.2004-10.com.ubuntu:01:52c55542e014 alias: vm-database
            Connection: 0
                IP Address: 192.168.56.104
    LUN information:
        LUN: 0
            Type: controller
            SCSI ID: IET     00010000
            SCSI SN: beaf10
            Size: 0 MB, Block size: 1
            Online: Yes
            Removable media: No
            Prevent removal: No
            Readonly: No
            SWP: No
            Thin-provisioning: No
            Backing store type: null
            Backing store path: None
            Backing store flags:
        LUN: 1
            Type: disk
            SCSI ID: IET     00010001
            SCSI SN: beaf11
            Size: 2147 MB, Block size: 512
            Online: Yes
            Removable media: No
            Prevent removal: No
            Readonly: No
            SWP: No
            Thin-provisioning: No
            Backing store type: rdwr
            Backing store path: /mnt/iscsi-disk.img
            Backing store flags:
    Account information:
    ACL information:
        192.168.56.0/24
ayoub@vm-applicatif:~/mon-app$
```

Figure 5.5 – *Serveur SAN iSCSI — Target avec LUN de 2147 MB*

5.4.2 Client iSCSI sur vm-database

```
vm-database (Instantané 2) [En fonction] - Oracle VirtualBox
Fichier  Machine  Écran  Entrée  Périphériques  Aide
ayoub@vm-database:~$ lsblk | grep sdb
sdb                         8:16   0    2G  0 disk /mnt/san
ayoub@vm-database:~$ df -h | grep san
/dev/sdb                      2,0G   28K  1,8G   1% /mnt/san
ayoub@vm-database:~$ _
```

Figure 5.6 – *Client SAN — Disque iSCSI /dev/sdb (2 GB) monté sur /mnt/san*

```
ayoub@vm-database:~$ lsblk | grep sdb
sdb     8:16    0     2G   0 disk /mnt/san

ayoub@vm-database:~$ df -h | grep san
/dev/sdb     2.0G     28K     1.8G     1%     /mnt/san
```

Listing 5.2 – *Vérification SAN iSCSI sur vm-database*

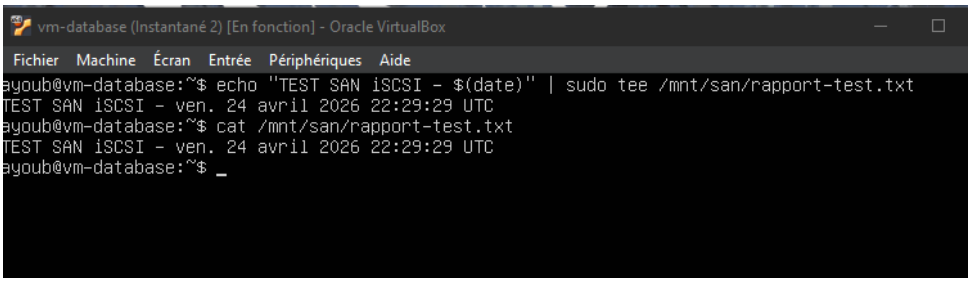


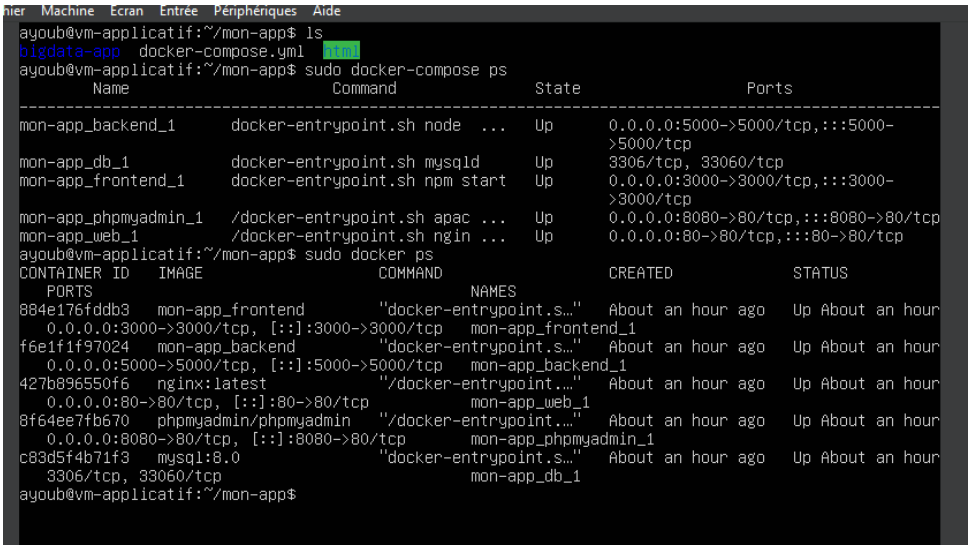
Figure 5.7 – *Test SAN — Écriture et lecture sur /mnt/san*

Montage automatique au démarrage : Le NAS NFS et le SAN iSCSI sont configurés pour se monter automatiquement via `/etc/fstab` et les services `open-iscsi` et `iscsid` activés avec systemctl.

Chapitre 6 — Containerisation et Application Big Data

6.1 Architecture Docker

La stack applicative est déployée via **Docker Compose** sur vm-applicatif. Cinq containers interconnectés via le réseau bridge *app-network* forment une application Big Data complète.



```
ayoub@vm-applicatif:~/mon-app$ ls
bigdata-app  docker-compose.yml  html
ayoub@vm-applicatif:~/mon-app$ sudo docker-compose ps
        Name                      Command               State                  Ports
-----------------------------------------------------------------------------------------------------
mon-app_backend_1      docker-entrypoint.sh node  ...   Up      0.0.0.0:5000->5000/tcp,:::5000-
                                                                >5000/tcp
mon-app_db_1           docker-entrypoint.sh mysqld      Up      3306/tcp, 33060/tcp
mon-app_frontend_1     docker-entrypoint.sh npm start   Up      0.0.0.0:3000->3000/tcp,:::3000-
                                                                >3000/tcp
mon-app_phpmyadmin_1   /docker-entrypoint.sh apac ...   Up      0.0.0.0:8080->80/tcp,:::8080->80/tcp
mon-app_web_1          /docker-entrypoint.sh ngin ...   Up      0.0.0.0:80->80/tcp,:::80->80/tcp
ayoub@vm-applicatif:~/mon-app$ sudo docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED             STATUS
   PORTS                                          NAMES
884e176fddb3   mon-app_frontend         "docker-entrypoint.s…"   About an hour ago   Up About an hour
   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp    mon-app_frontend_1
f6e1f1f97024   mon-app_backend          "docker-entrypoint.s…"   About an hour ago   Up About an hour
   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp    mon-app_backend_1
427b896550f6   nginx:latest             "/docker-entrypoint.…"   About an hour ago   Up About an hour
   0.0.0.0:80->80/tcp, [::]:80->80/tcp            mon-app_web_1
8f64ee7fb670   phpmyadmin/phpmyadmin    "/docker-entrypoint.…"   About an hour ago   Up About an hour
   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp        mon-app_phpmyadmin_1
c83d5f4b71f3   mysql:8.0                "docker-entrypoint.s…"   About an hour ago   Up About an hour
   3306/tcp, 33060/tcp                            mon-app_db_1
ayoub@vm-applicatif:~/mon-app$
```

Figure 6.1 – *Containers Docker actifs (docker-compose ps)*

Table 6.1 – *Stack Docker déployée*

tableheadbg			
mon-app_web_1	nginx :latest	**80**	Serveur web
tablerowalt mon-app_frontend_1	node :18-alpine (React)	**3000**	Dashboard utilisateur
mon-app_backend_1	node :18-alpine (Node.js)	**5000**	API REST Big Data
tablerowalt mon-app_db_1	mysql :8.0	**3306**	Base de données
mon-app_phpmyadmin_1	phpmyadmin/phpmyadmin	**8080**	Administration DB

6.2 Application Big Data — Architecture 3-Tiers

L'application suit une architecture **3-tiers** : présentation (React.js), logique métier (Node.js + Express) et données (MySQL).

6.2.1 Page d'accueil Nginx

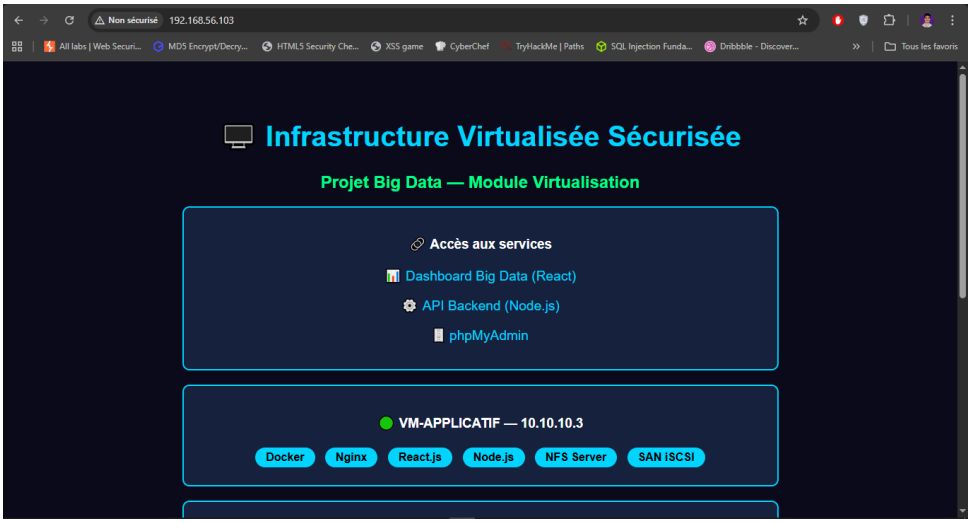


Figure 6.2 – *Page d'accueil Nginx avec liens vers tous les services*

6.2.2 Dashboard React.js

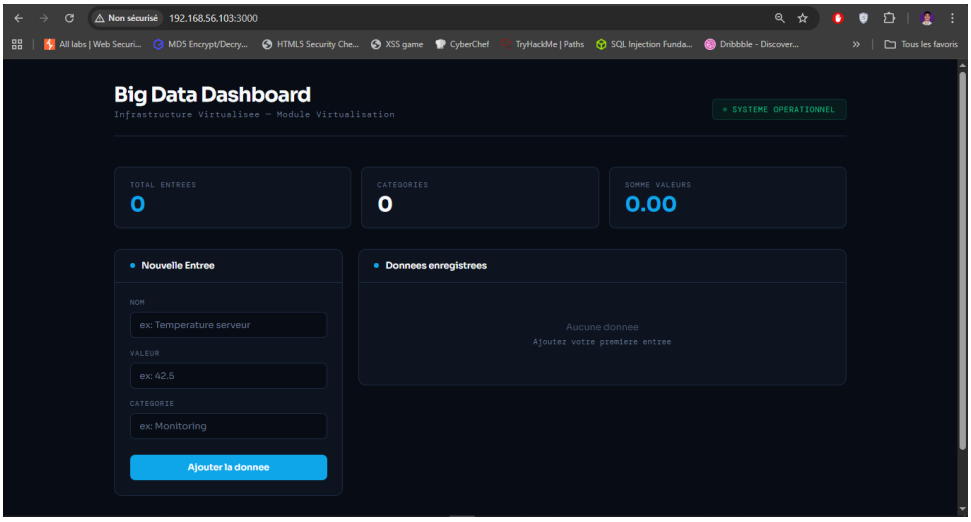


Figure 6.3 – *Dashboard Big Data — Interface React.js professionnelle*

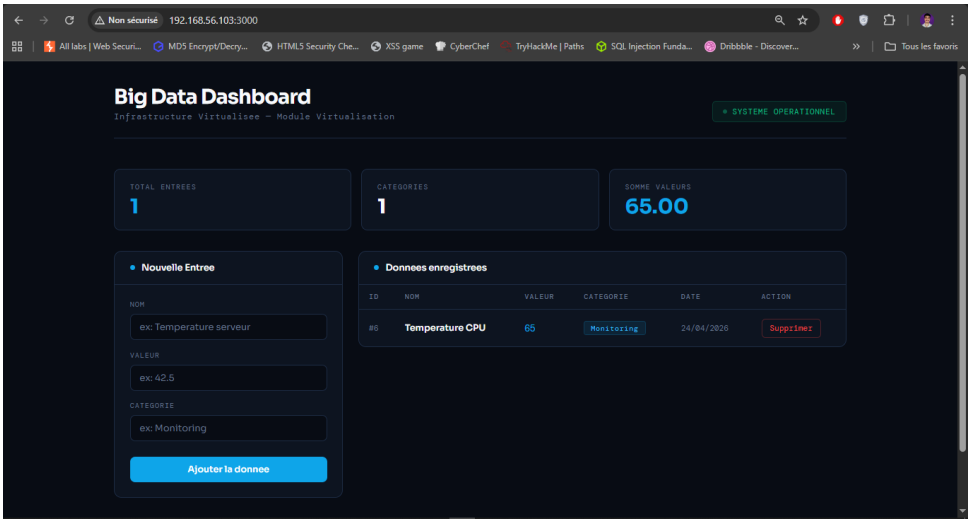


Figure 6.4 – *Ajout d'une donnée Big Data via le formulaire React.js*

6.2.3 API Backend Node.js

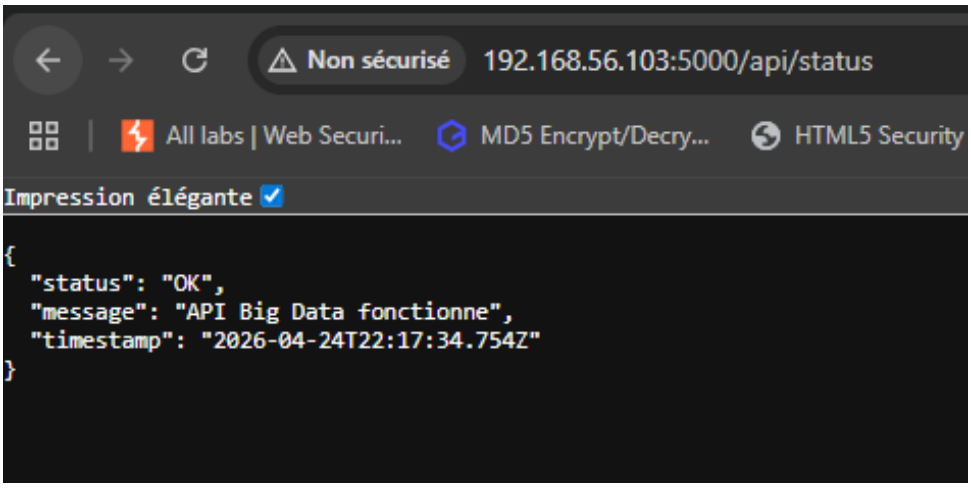


Figure 6.5 – *API Backend Node.js — Endpoint /api/status*

```
GET http://192.168.56.103:5000/api/status
{
  "status": "OK",
  "message": "API Big Data fonctionne",
  "timestamp": "2026-04-24T17:52:01.268Z"
}
```

Listing 6.1 – *Réponse de l'API Backend*

6.2.4 Interface phpMyAdmin

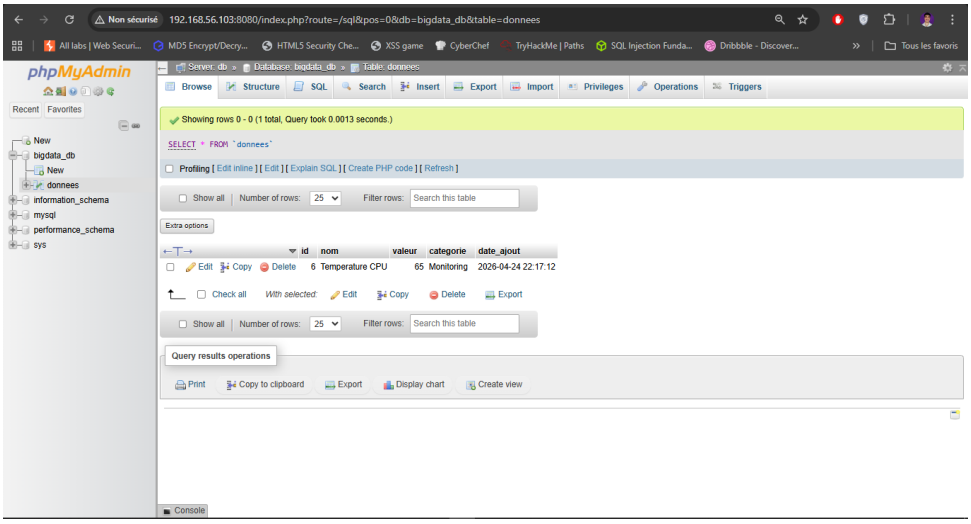


Figure 6.6 – *phpMyAdmin — Visualisation de bigdata_db et de la table donnees*

6.2.5 Logs Backend

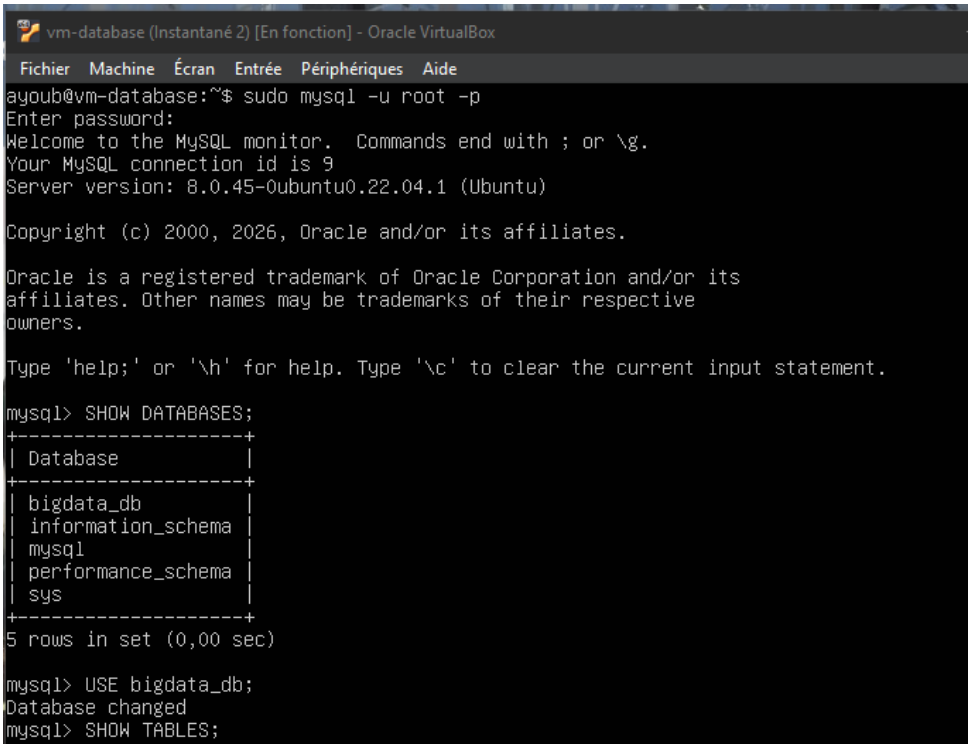


Figure 6.7 – *Logs du backend Node.js — Connexion réussie à MySQL*

6.3 Démarrage Automatique avec systemd

Un service systemd assure le démarrage automatique des containers Docker au démarrage de vm-applicatif, sans intervention manuelle.

```
[Unit]
Description=BigData Docker App
Requires=docker.service
After=docker.service network.target

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/home/ayoub/mon-app
ExecStart=/usr/bin/docker-compose up -d
ExecStop=/usr/bin/docker-compose down
User=root

[Install]
```

```
WantedBy=multi-user.target
```

Listing 6.2 – *Service systemd bigdata-app.service*

Chapitre 7 — Sécurité de l'Infrastructure

7.1 Approche Multi-Couches

La sécurité de l'infrastructure repose sur une approche **defense in depth** combinant plusieurs niveaux de protection indépendants.

Table 7.1 – *Mesures de sécurité implémentées*

tableheadbg		
Réseau	VLANs isolés	Segmentation et isolation inter-VMs
tablerowalt Réseau	UFW Firewall	Filtrage des ports par VM
Accès	Bastion Host	Point d'entrée SSH unique
tablerowalt Accès	SSH RSA 4096	Chiffrement asymétrique sans mot de passe
Données	mysql_secure_installation	Sécurisation de MySQL
tablerowalt Données	Backup automatique	Sauvegarde quotidienne via cron

7.2 Chiffrement SSH RSA 4096

Des paires de clés RSA 4096 bits ont été générées et distribuées entre les VMs, permettant des connexions SSH **sans mot de passe** via cryptographie asymétrique.

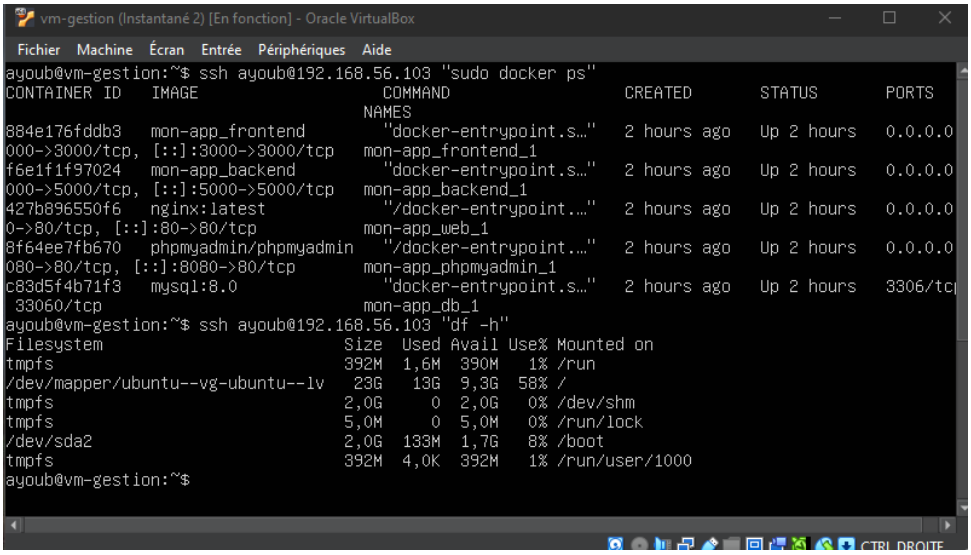


Figure 7.1 – *SSH sans mot de passe depuis vm-gestion vers les autres VMs*

7.3 Sécurisation MySQL

```
Remove anonymous users?  [Y] --> Utilisateurs anonymes supprimes
Disallow root login remotely? [N] --> Conserve pour administration
Remove test database? [Y] --> Base test supprimee
Reload privilege tables? [Y] --> Tables rechargees
All done!
```

Listing 7.1 – *Résultat de mysql_secure_installation*

7.4 Contrôle d'Accès par VLAN

Grâce aux VLANs, vm-database est **inaccessible directement depuis internet**. Elle ne peut être jointe que via les ports autorisés (3306 pour MySQL depuis le réseau local, et 22 uniquement depuis vm-gestion).

Chapitre 8 — Administration et Monitoring

8.1 Rôles de vm-gestion

vm-gestion constitue le **serveur d'administration centralisée** de l'infrastructure, assumant quatre rôles complémentaires :

1. **Bastion Host :** Point d'accès SSH unique et sécurisé.
2. **Monitoring :** Surveillance de l'état des services à distance.
3. **Administration :** Contrôle des VMs via SSH sans mot de passe.
4. **Sauvegarde :** Backup automatique MySQL planifié via cron.

8.2 Monitoring à Distance

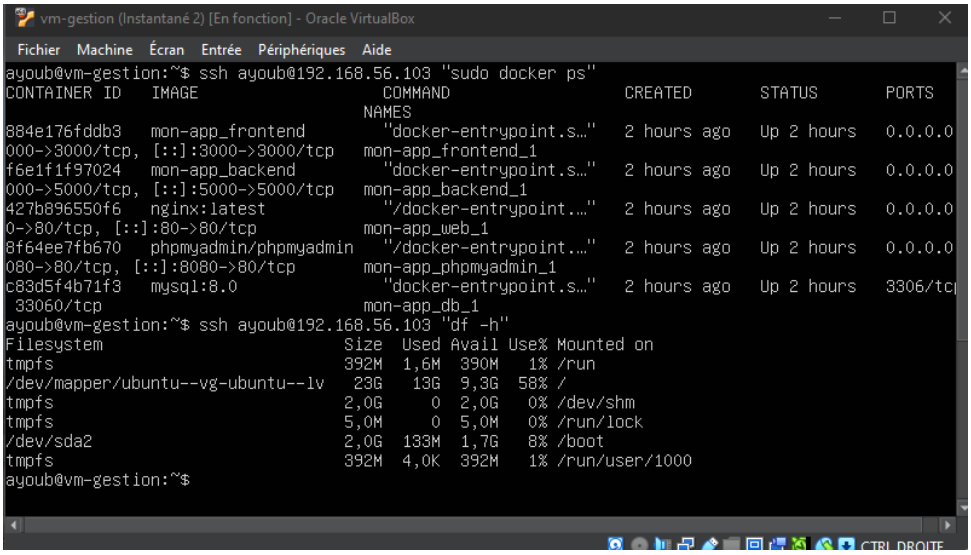


Figure 8.1 – *Monitoring depuis vm-gestion — Docker, MySQL, disque, RAM*

```
# Verifier les containers Docker sur vm-applicatif
ssh ayoub@192.168.56.103 "sudo docker ps | grep Up"

# Verifier l'etat de MySQL sur vm-database
ssh ayoub@192.168.56.104 "sudo systemctl is-active mysql"
active

# Surveiller l'espace disque de vm-applicatif
ssh ayoub@192.168.56.103 "df -h"

# Surveiller la RAM de vm-database
ssh ayoub@192.168.56.104 "free -h"
```

Listing 8.1 – *Commandes de monitoring depuis vm-gestion*

8.3 Sauvegarde Automatique

```
ayoub@vm-gestion:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command

0 2 * * * ssh ayoub@192.168.56.104 "sudo mysqldump bigdata_db" > /mnt/nas/backup/backup-$(date +\%F)
.sql

# Vérification services toutes les 5 minutes
*/5 * * * * ssh ayoub@192.168.56.103 "sudo docker ps | grep Up" >> /mnt/nas/logs/docker-status.log
*/5 * * * * ssh ayoub@192.168.56.104 "sudo systemctl is-active mysql" >> /mnt/nas/logs/mysql-status.
log
ayoub@vm-gestion:~$
```

Figure 8.2 – *Sauvegarde automatique MySQL dans /mnt/nas/backup/*

```
# Crontab de vm-gestion (crontab -l)
0 2 * * * ssh ayoub@192.168.56.104 \
  "sudo mysqldump bigdata_db" \
  > /mnt/nas/backup/backup-$(date +\%F).sql

# Resultat dans /mnt/nas/backup/
ls -lh /mnt/nas/backup/
-rw-rw-r-- 1 ayoub ayoub 1.3K avril 23 22:45 backup-2026-04-23.
    sql
```

Listing 8.2 – *Tâche cron de sauvegarde automatique quotidienne*

Chapitre 9 — Difficultés Rencontrées et Solutions

9.1 Difficulté 1 — Erreur d'Installation Ubuntu

Problème : Lors de l'installation d'Ubuntu Server sur VirtualBox, une erreur *"installing kernel"* survenait à l'étape de téléchargement du noyau, causée par un problème de miroir réseau.

Solution apportée : Désactivation du miroir réseau pendant l'installation pour utiliser exclusivement l'ISO locale. L'installation s'est terminée avec succès.

9.2 Difficulté 2 — Conflit d'IPs après Clonage

Problème : Après le clonage des VMs, vm-database et vm-gestion avaient la même IP Host-Only (`192.168.56.104`), causant des conflits de communication.

Solution apportée : Configuration d'IPs statiques distinctes via Netplan et désactivation du serveur DHCP VirtualBox dans le Gestionnaire de réseau hôte.

9.3 Difficulté 3 — Disque vm-applicatif Saturé

Problème : Le disque de vm-applicatif atteignait 97% d'utilisation suite à l'installation des images Docker et du fichier iSCSI de 2 GB.

Solution apportée : Extension du disque VDI de 25 GB à 60 GB via le Gestionnaire de médias VirtualBox, puis agrandissement à chaud du volume LVM :

```
sudo growpart /dev/sda 3
sudo lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
sudo resize2fs /dev/mapper/ubuntu--vg-ubuntu--lv
# Resultat : 97% --> 48% d'utilisation
```

Listing 9.1 – *Extension du disque LVM à chaud*

9.4 Difficulté 4 — Erreur ContainerConfig Docker Compose

Problème : Docker Compose 1.29.2 générait une erreur *KeyError : 'ContainerConfig'* lors de la recréation de containers, due à une incompatibilité avec les nouvelles versions d'images Docker.

Solution apportée : Suppression complète des containers et images existants, puis reconstruction depuis zéro :

```
sudo docker-compose down
sudo docker system prune -f
sudo docker-compose up -d --build
```

Listing 9.2 – *Résolution de l'erreur ContainerConfig*

9.5 Difficulté 5 — SAN iSCSI Non Monté au Démarrage

Problème : Le disque iSCSI `/dev/sdb` n'était pas monté automatiquement après redémarrage de vm-database, nécessitant une intervention manuelle à chaque démarrage.

Solution apportée : Activation des services iSCSI au démarrage, configuration du montage automatique dans `/etc/fstab` et configuration du démarrage automatique iSCSI :

```
sudo systemctl enable open-iscsi
sudo systemctl enable iscsid
sudo iscsiadm -m node -o update -n node.startup -v automatic
# Ajout dans /etc/fstab :
# /dev/sdb /mnt/san ext4 defaults,_netdev,auto 0 0
```

Listing 9.3 – *Configuration du montage automatique SAN*

9.6 Difficulté 6 — Backend Node.js Ne Se Connecte Pas à MySQL

Problème : Le container Node.js tentait de se connecter à MySQL avant que ce dernier ne soit complètement démarré, générant une erreur *ECONNREFUSED*.

Solution apportée : Implémentation d'un mécanisme de **retry automatique** dans le code Node.js, attendant 5 secondes entre chaque tentative de connexion jusqu'à ce que MySQL soit disponible.

Chapitre 10 — Analyse Critique

10.1 Apports de la Virtualisation

10.1.1 Optimisation des ressources

La virtualisation a permis de faire fonctionner trois serveurs distincts sur une seule machine physique, avec une réduction significative des coûts matériels. Le Thin Provisioning optimise l'utilisation du stockage : un disque virtuel de 60 GB n'occupe que 11 GB réellement.

10.1.2 Sécurité renforcée

L'architecture **defense in depth** combine VLANs, UFW, Bastion Host et SSH RSA 4096, créant plusieurs barrières indépendantes. La compromission de vm-applicatif ne donne pas automatiquement accès à vm-database.

10.1.3 Flexibilité et résilience

Les snapshots permettent une restauration en quelques minutes. Les sauvegardes automatiques MySQL garantissent la continuité des données. Le montage automatique NFS/iSCSI assure la disponibilité des services sans intervention manuelle.

10.2 Apports de la Containerisation

Docker Compose simplifie considérablement le déploiement de la stack applicative. Les containers offrent isolation, reproductibilité et scalabilité. Le service systemd assure un démarrage automatique fiable.

Chapitre 11 — Conclusion

Ce projet a permis de concevoir et mettre en œuvre une infrastructure virtualisée complète, fonctionnelle et sécurisée, couvrant l'ensemble des objectifs du module de Technologies de Virtualisation.

Bilan technique de l'infrastructure réalisée :

- ✓ Hyperviseur VirtualBox avec 3 VMs Ubuntu Server (Thin Provisioning, LVM, Snapshots)
- ✓ Segmentation réseau en 3 VLANs isolés avec UFW Firewall et Bastion Host SSH
- ✓ Stockage virtualisé 3 niveaux : DAS (VDI/LVM), NAS (NFS), SAN (iSCSI) avec auto-mount
- ✓ Stack Big Data containerisée : React.js, Node.js, MySQL, Nginx, phpMyAdmin via Docker Compose
- ✓ Administration centralisée : monitoring, backup automatique quotidien, SSH RSA 4096
- ✓ Démarrage automatique complet via systemd et fstab

Sur le plan **pédagogique**, ce projet a consolidé les compétences en virtualisation serveur, réseau et stockage, ainsi qu'en containerisation Docker et développement d'applications web modernes. Il a permis de confronter les concepts théoriques à leur mise en pratique concrète, avec les difficultés et solutions inhérentes à tout déploiement réel.

L'infrastructure est entièrement opérationnelle, reproductible et documentée, constituant une base solide pour des déploiements en environnement professionnel.

Chapitre 11 — Références Bibliographiques

1. Oracle Corporation. *VirtualBox User Manual*. https://www.virtualbox.org/manual/, 2024.

2. Canonical Ltd. *Ubuntu Server Guide — 22.04 LTS*. https://ubuntu.com/server/docs, 2024.

3. Docker Inc. *Docker Documentation*. https://docs.docker.com/, 2024.

4. Oracle Corporation. *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/refman/8.0/en/, 2024.

5. Meta Open Source. *React Documentation*. https://react.dev/, 2024.

6. OpenJS Foundation. *Node.js Documentation*. https://nodejs.org/en/docs/, 2024.

7. S. Shepler et al. *Network File System (NFS) Version 4 Protocol*. RFC 7530, IETF, 2015.

8. J. Satran et al. *Internet Small Computer Systems Interface (iSCSI)*. RFC 3720, IETF, 2004.

9. Canonical Ltd. *UFW — Uncomplicated Firewall*. https://help.ubuntu.com/community/UFW, 2024.

10. Slides de cours. *Module Technologies de Virtualisation — Master Introduction à la Virtualisation*. Année 2025–2026.